

Implementation and Comparison of A* Search and Greedy Best First Search Algorithms for Path Finding

Name: VIKASH RAJ PATEL

Registration Number: 25MIB10051

Abstract

This project demonstrates the implementation of two informed search algorithms in artificial intelligence: A* Search and Greedy Best First Search. The objective is to explore how heuristic information guides the search process in a grid based path finding problem. The project highlights differences in optimality, search behavior, path length, and the number of explored nodes between the two methods.

Introduction

Search algorithms are a core component of artificial intelligence and are used in problem solving agents. Search strategies can be classified as uninformed and informed search methods.

In this project, A* Search and Greedy Best First Search are implemented as examples of informed search techniques. The algorithms are applied to a grid based environment where the goal is to find a path from a start node to a target node while avoiding blocked cells.

Objectives

- To understand the working of A* Search and Greedy Best First Search algorithms
- To implement both algorithms using Python
- To use Manhattan distance as a heuristic for grid based path finding
- To compare path length and the number of explored nodes

Methodology

1. Problem Representation

- A grid is used to represent the environment
- Each cell is treated as a node
- Movement is allowed in four directions
- Blocked cells act as obstacles

2. A* Search

- Uses a priority queue
- Uses the evaluation function $f(n) = g(n) + h(n)$
- $g(n)$ represents the path cost from the start node
- $h(n)$ represents the estimated distance to the goal
- Uses Manhattan distance as the heuristic

3. Greedy Best First Search

- Uses a priority queue
- Uses the evaluation function $f(n) = h(n)$
- Selects the node that appears closest to the goal
- Does not guarantee the shortest path

Implementation

A* Search Logic

- Initialize the priority queue with the start node
- Track the cost from the start node to each visited node

- Choose nodes using $g(n) + h(n)$
- Reconstruct the path after reaching the goal

Greedy Best First Search Logic

- Initialize the priority queue using the heuristic value
- Choose the node with the lowest $h(n)$ value
- Continue until the goal is reached
- Reconstruct the path using stored parent nodes

Results and Discussion

Feature	A* Search	Greedy Best First Search
Evaluation Function	$g(n) + h(n)$	$h(n)$
Path Cost	Considered	Not considered
Heuristic	Used	Used
Optimality	Yes, with an admissible heuristic	Not guaranteed
Path Length in Demo	12 steps	14 steps
Nodes Explored in Demo	37	17

In the included demonstration grid, A* Search finds a 12-step path while Greedy Best First Search finds a 14-step path. Greedy explores fewer nodes in this example because it follows the heuristic more aggressively, but the resulting path is longer. A* considers both travelled cost and estimated remaining cost, allowing it to find the shorter path.

Applications

- Path finding in maps and navigation systems
- Game AI and movement planning
- Robot navigation
- Problem solving agents

Conclusion

The project successfully demonstrates the implementation of A* Search and Greedy Best First Search. It shows how heuristic information affects the search process. A* combines the travelled path cost and estimated remaining cost, while Greedy Best First Search focuses only on estimated closeness to the goal. The comparison demonstrates that Greedy search may explore fewer nodes, but A* is more reliable when an optimal path is required.

References

- Artificial Intelligence: A Modern Approach by Russell and Norvig
- Course Material: Fundamentals in AI and ML